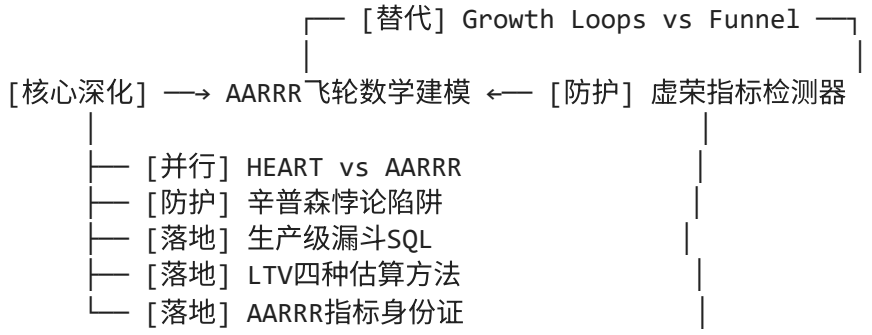
拓展：AARRR的里子——从「画漏斗」到「数学建模+代码落地」

> **本拓展讲什么**：主课教你 AARRR 五环节框架和业务诊断。本拓展往深处走——把 AARRR 的飞轮用数学建模、用 SQL 落地到生产、用代码检测「你选的指标是不是虚荣指标」。 **适合谁**：刚学完 W2D1 主课，能用 AARRR 画漏斗做诊断，但想知道「怎么把 AARRR 变成可跑的生产代码」和「AARRR 选指标有哪些坑」。 **预计时长**：约 2-2.5h

---

知识网络预览



本拓展新增	8 个课程未覆盖的深度概念
[必修] 6个	飞轮建模、HEART对比、虚荣指标检测、辛普森陷阱、漏斗SQL、Growth Loops
[选修] 2个	LTV四种算法、AARRR指标身份证

第一部分：沙盘设定

快达出行（第 6 次出场）

当前状态：上周末你帮快达做了异动分析（W1D6）。这周一，CEO 给了你一个新任务——

> 「我们过去半年 DAU 从 50 万涨到了 120 万，但付费收入只涨了 30%。增长团队说『一切正常，DAU 在涨』——但我觉得不对劲。你帮我用 AARRR 从头到尾查一遍，告诉我到底卡在哪。」

你的角色：快达出行的增长数据分析师。你有完整的数据权限。

数据资产（≥8 张核心表）：

- `ods_order_di` — 订单明细（order_id, user_id, driver_id, city, amount, status, create_time）
- `dwd_user_register_di` — 用户注册表（user_id, channel, city, register_time）
- `dwd_user_active_di` — 用户日活表（user_id, pv, session_cnt, core_action_cnt, dt）
- `dwd_trip_core_di` — 行程核心事件表（user_id, trip_id, event_type, event_time）
- `dwd_payment_di` — 支付明细（user_id, order_id, amount, pay_time）
- `dwd_invite_log_di` — 邀请记录（inviter_id, invitee_id, invite_time, status）
- `dim_channel_info` — 渠道维表（channel_id, channel_name, channel_type, cost_per_click）
- `dim_user_label` — 用户标签表（user_id, label_type, label_value）
- `ads_user_cohort_retention` — 同期群留存汇总（cohort_date, day_n, retention_rate）

第二部分：入口自诊

Q1：AARRR 五个环节中，哪个环节的改善对最终收入的放大效应最大？为什么？

Q2：激活和留存的核心区别是什么？如果激活率很高（60%）但次日留存很低（10%），说明什么？

Q3：K 因子 = 0.5 意味着什么？如果 K 因子突然从 0.5 降到 0.2，你应该最先查 AARRR 的哪个环节？

自诊反馈

题号	✅ 参考答案	❌ 如果答偏了
Q1	下游环节（激活/留存/变现）的放大效应 > 上游（获取）。因为每多一个激活用户，他会走完留存→变现→推荐全链路	说「都一样」→回去看主课 2.6 节的代码输出
Q2	激活 = 第一次体验核心价值；留存 = 持续回来。激活率高但留存低 = 产品有「啊哈时刻」但没有「粘性」——用户第一次觉得好，但没理由第二天再回来	混淆激活和留存→回看主课 1.1 节定义
Q3	K=0.5 意味着每个用户平均带来 0.5 个新用户。K 从 0.5→0.2 意味着推荐急剧下降	直接查推荐→可能忽略了上游的恶化

题号	✅ 参考答案	❌ 如果答偏了
	——最先应该查「留存是否在恶化」	

第三部分：概念网络展开

🔴 核心深化：AARRR 飞轮的数学建模

> 主课用文字和示意图描述了飞轮效应。但这个飞轮到底能不能「自转」——需要数学来回答。

🔴 认知冲突

快达的 CEO 问你：「如果我们把 K 因子从 0.2 提升到 0.4，能少投多少广告？」

你的直觉：K 翻倍 → 推荐带来的用户翻倍 → 投放可以减半？

算一下：假设月投放带来 50 万新用户，K=0.2 时推荐额外带来 50/(1-0.2) - 50 = 12.5 万。K=0.4 时推荐带来 50/(1-0.4) - 50 = 33.3 万。推荐用户增长了 20.8 万，但远不到「投放减半」的程度。

> 飞轮不是「推荐替代投放」——是「推荐放大投放的效率」。只有 K>1 时才是真正的「免费增长」。

[必修] AARRR 飞轮的数学模型

① **一句话白话**：AARRR 飞轮 = 投放 → 用户 → 体验好 → 推荐 → 更多用户。飞轮转速由两个参数决定：转化率乘积（飞轮效率），以及 K 因子（飞轮放大倍数）。

② **核心逻辑**：

设每月投放获取用户数为 A_0，推荐 K 因子为 k（每个用户平均带来 k 个新用户）。

总用户 = $A_0 \times (1 + k + k^2 + k^3 + ...) = A_0 / (1 - k)$，当 $k < 1$ 时收敛。

这个公式告诉你三件事：

1. k=0.2 时，总用户 = $A_0/0.8 = 1.25A_0$（投放放大 25%）
2. k=0.5 时，总用户 = $A_0/0.5 = 2A_0$（投放放大 100%）
3. k→1 时，总用户→∞（病毒式增长——现实中极少见）

③ **飞轮的「转速」**：推荐有延迟。用户从激活→推荐平均需要 T 天。T 越短，飞轮转得越快。有效 K = k / T。

In [1]:
```python
# [必修] AARRR 飞轮建模：K因子的增长放大效应
import numpy as np

A0 = 50000
k_values = [0.1, 0.2, 0.3, 0.5, 0.8, 0.95]

print("=" * 60)
print("快达出行 AARRR 飞轮建模")
print("=" * 60)
print()
print("【实验1】K因子对总用户的放大倍数")
print(f"{'K因子':<10} {'总用户/月':>12} {'投放放大倍数':>12} {'推荐占比':>10}")
print("-" * 50)

for k in k_values:
    total_users = A0 / (1 - k)
    multiplier = 1 / (1 - k)
    referral_pct = (total_users - A0) / total_users * 100
    print(f"{k:<10} {total_users:>12,.0f} {multiplier:>11.2f}x {referral_pct:>9.

print()
print("关键发现：")
print(f"  K=0.2（行业平均）-> 投放放大仅 1.25x -> 推荐只贡献 20% 用户")
print(f"  K=0.5（优秀水平）-> 投放放大 2.00x -> 推荐贡献 50% 用户")
print(f"  K=0.8（极少数产品）-> 投放放大 5.00x -> 推荐贡献 80% 用户")
```

```
============================================================
快达出行 AARRR 飞轮建模
============================================================

【实验1】K因子对总用户的放大倍数
K因子                 总用户/月        投放放大倍数        推荐占比
--------------------------------------------------
0.1              55,556        1.11x      10.0%
0.2              62,500        1.25x      20.0%
0.3              71,429        1.43x      30.0%
0.5             100,000        2.00x      50.0%
0.8             250,000        5.00x      80.0%
0.95          1,000,000       20.00x      95.0%

关键发现：
  K=0.2（行业平均）-> 投放放大仅 1.25x -> 推荐只贡献 20% 用户
  K=0.5（优秀水平）-> 投放放大 2.00x -> 推荐贡献 50% 用户
  K=0.8（极少数产品）-> 投放放大 5.00x -> 推荐贡献 80% 用户
```

In [2]:
```python
# === 飞轮转速：引入时间延迟 ===
months = np.arange(0, 13)
k = 0.3
users_by_month = np.zeros(len(months))
users_by_month[0] = A0

for m in range(1, len(months)):
    users_by_month[m] = users_by_month[m-1] * k

print("=" * 60)
print("【实验2】飞轮转速：K因子的传播时间延迟")
```

```
print("=" * 60)
print(f"K={k}，每月纯投放={A0:,} 用户")
print(f"{'月份':<8} {'当月新增':>20} {'推荐占比':>10}")
print("-" * 42)
for m in range(len(months)):
    ref_pct = 0 if m == 0 else users_by_month[m] / (A0 + users_by_month[m]) * 10
    print(f"第{m}月    {A0 + users_by_month[m]:>20,.0f} {ref_pct:>9.1f}%")

steady_state = 1 / (1 - k)
print(f"\n稳态放大倍数: {steady_state:.2f}x")
print(f"达到稳态 95% 约需: {int(np.log(0.05) / np.log(k))} 个月")
print("结论: K 越小，飞轮加速越慢")
```

```
============================================================
【实验2】飞轮转速：K因子的传播时间延迟
============================================================
K=0.3，每月纯投放=50,000 用户
月份                          当月新增        推荐占比
------------------------------------------
第0月                  100,000       0.0%
第1月                   65,000      23.1%
第2月                   54,500       8.3%
第3月                   51,350       2.6%
第4月                   50,405       0.8%
第5月                   50,122       0.2%
第6月                   50,036       0.1%
第7月                   50,011       0.0%
第8月                   50,003       0.0%
第9月                   50,001       0.0%
第10月                   50,000       0.0%
第11月                   50,000       0.0%
第12月                   50,000       0.0%

稳态放大倍数: 1.43x
达到稳态 95% 约需: 2 个月
结论: K 越小，飞轮加速越慢
```

In [3]:

```
# === 各环节改进对K因子的贡献分解 ===
baseline = {'invites_per_user': 1.5, 'invite_conversion': 0.20,
            'activation': 0.40, 'retention_d30': 0.15}
baseline_k = baseline['invites_per_user'] * baseline['invite_conversion']

print("=" * 60)
print("【实验3】各环节改进对K因子的贡献分解")
print("=" * 60)
print(f"基线 K = {baseline_k}\n")

improvements = {
    '邀请次数+20%': ('invites_per_user', 1.20),
    '邀请转化+20%': ('invite_conversion', 1.20),
    '激活率+20%(40%->48%)': ('activation', 1.20),
    '月留存+20%(15%->18%)': ('retention_d30', 1.20),
}

for name, (key, factor) in improvements.items():
    invites = baseline['invites_per_user'] * (factor if key == 'invites_per_user'
    conv = baseline['invite_conversion'] * (factor if key == 'invite_conversion'
    if key in ('activation', 'retention_d30'):
        invites *= factor
    new_k = invites * conv
```

```
    print(f"{name:<30}: K = {baseline_k:.2f} -> {new_k:.2f} (Delta = {new_k - ba

print()
print("关键发现：激活和留存的改善，间接提升了推荐—")
print("  它们让更多用户活到愿意推荐的阶段。这就是飞轮。")
```

```
============================================================
【实验3】各环节改进对K因子的贡献分解
============================================================
基线 K = 0.30000000000000004

邀请次数+20%                    : K = 0.30 -> 0.36 (Delta = +0.06)
邀请转化+20%                    : K = 0.30 -> 0.36 (Delta = +0.06)
激活率+20%(40%->48%)            : K = 0.30 -> 0.36 (Delta = +0.06)
月留存+20%(15%->18%)            : K = 0.30 -> 0.36 (Delta = +0.06)

关键发现：激活和留存的改善，间接提升了推荐—
  它们让更多用户活到愿意推荐的阶段。这就是飞轮。
```

概念群 A：[并行] HEART vs AARRR——什么时候用哪个？

> **来源**：全课程永久空白区 Section 5.3 | **深度类型**：[B] 案例深度

[必修] HEART vs AARRR 系统对比

① **一句话白话**：AARRR 是「增长视角」——问「用户怎么来、怎么留、怎么付钱」。HEART 是「体验视角」——问「用户用起来爽不爽、效率高不高」。增长团队用 AARRR，产品体验团队用 HEART。

② **HEART 五维度**（Google 的 UX 度量框架）：

维度	英文	衡量什么	典型指标	快达例子
满意度	Happiness	用户主观感受	NPS、App Store评分	乘客打完车后的评分
参与度	Engagement	用户卷入深度	人均使用时长、人均session数	司机日均在线时长
采用率	Adoption	新功能被多少人用	新功能使用率	OSM定义的策略渗透率
留存率	Retention	用户是否回来	D1/D7/D30留存	与AARRR的R完全一致
任务成功	Task Success	核心任务是否高效完成	任务完成率、完成时间	用户从打开App到上车的时间

③ **AARRR vs HEART 选择矩阵**：

场景	用 AARRR	用 HEART
增长策略制定	✅ 最适合	❌ 偏体验

场景	用 AARRR	用 HEART
产品改版评估	⚠️ 只能看增长影响	✅ 适合
用户研究	❌ 关注量不关注质	✅ 最适合
渠道ROI分析	✅ 最适合	❌ 不涉及渠道

④ **两个框架可以叠加**：AARRR 的每个环节可以用 HEART 的维度来度量。激活（AARRR）用 HEART 的「任务成功率」度量；留存用「参与度」度量；推荐用「满意度」度量。

⑤ **与核心概念的关系**：[并行] — AARRR 讲增长的五个环节，HEART 讲每个环节的体验质量。同时掌握 = 既知道漏在哪又知道为什么漏。

概念群 B：[防护] 虚荣指标检测器——AARRR 每个环节最常见的假指标

> **来源**：全课程永久空白区 Section 5.3 | **深度类型**：[C] 代码深度

🔴 认知冲突

快达的增长团队在周报里汇报：「本月新增用户 80 万，环比 +60%！增长形势大好！」

你翻开数据——这 80 万里，50 万来自「注册送 10 元优惠券」活动。这 50 万人中：

- 40% 注册后再也没有打开过 App
- 次日留存只有 3%（正常渠道 20%）
- 30 日内付费转化率 0.1%（正常渠道 5%）

> 「新增用户数」是一个虚荣指标——它在涨，但产品没有变好。增长团队在优化「指标」而不是「业务」。

In [4]:
```python
# [必修] 虚荣指标自动检测器
import numpy as np; np.random.seed(42)

class VanityMetricDetector:
    """检测逻辑：1.单调性检验 2.可操纵性检验 3.价值相关性检验"""

    def check_monotonicity(self, metric_series):
        diffs = np.diff(metric_series)
        up_ratio = np.mean(diffs > 0)
        max_streak = 0; current = 0
        for d in diffs:
            current = current + 1 if d > 0 else 0
            max_streak = max(max_streak, current)
        score, reasons = 0, []
        if up_ratio > 0.70: score += 25; reasons.append(f"上涨天数占比 {up_ratio:
        if max_streak > 10: score += 25; reasons.append(f"最长连涨 {max_streak} 
        return {'score': min(score,50), 'reasons': reasons, 'up_ratio': up_ratio
```

```
    def check_manipulability(self, metric_series):
        pct = np.abs(np.diff(metric_series) / (metric_series[:-1] + 1e-10))
        jumps = np.sum(pct > 0.5); max_jump = np.max(pct) if len(pct) > 0 else 0
        score, reasons = 0, []
        if jumps > 3: score += 15; reasons.append(f"{jumps}次超过50%单日跳变")
        if max_jump > 2.0: score += 10; reasons.append(f"最大单日变化 {max_jump:.
        return {'score': min(score,25), 'reasons': reasons}

    def check_value_correlation(self, metric, north_star):
        corr = np.corrcoef(metric, north_star)[0,1]
        score, reasons = 0, []
        if np.isnan(corr): return {'score':0,'reasons':['数据不足'],'correlation'
        if corr < 0.3: score += 25; reasons.append(f"与北极星r={corr:.2f} (<0.3)
        elif corr < 0.5: score += 10; reasons.append(f"与北极星弱相关r={corr:.2f}
        return {'score': score, 'reasons': reasons, 'correlation': corr}

    def detect(self, name, series, north_star=None):
        mono = self.check_monotonicity(series)
        manip = self.check_manipulability(series)
        value = self.check_value_correlation(series, north_star) if north_star i
        total = mono['score'] + manip['score'] + value['score']
        level = '健康' if total<=25 else ('关注' if total<=50 else ('疑似虚荣' if
        return {'metric':name, 'vanity_score':total, 'level':level,
                'reasons': mono['reasons']+manip['reasons']+value['reasons']}

# === 测试 ===
days = np.arange(90)
normal_dau = 100000 + np.sin(days/7*2*np.pi)*15000 + np.random.randn(90)*5000 +
cumulative = 10000 + days*500 + np.random.randn(90)*100
suspicious = np.ones(90)*100000; suspicious[30:35] = 200000
suspicious += np.random.randn(90)*8000

detector = VanityMetricDetector()
print("=" * 60)
print("虚荣指标自动检测器 -- 验证三个案例")
print("=" * 60)
for name, series, ns in [("正常DAU",normal_dau,normal_dau),
    ("累计注册用户数",cumulative,normal_dau),("被活动污染的新增",suspicious,normal_
    r = detector.detect(name, series, north_star=ns)
    print(f"\n【{r['metric']}】虚荣度评分: {r['vanity_score']}/100 -> {r['level']
    for reason in r['reasons']: print(f"  ! {reason}")

print("\n" + "=" * 60)
print("AARRR各环节虚荣指标 vs 健康指标")
print("=" * 60)
for stage, vanity, healthy in [
    ('获取','新增用户数','有效新增用户数(有核心行为)'),
    ('激活','注册完成率','首日核心行为完成率'),
    ('留存','MAU(月活)','同期群D30留存率'),
    ('变现','GMV(总交易额)','毛利率/净收入'),
    ('推荐','邀请发送量','邀请->注册转化率 x 被邀请人留存')]:
    print(f"{stage}: {vanity} -> {healthy}")
```

```
============================================================
虚荣指标自动检测器 -- 验证三个案例
============================================================

【正常DAU】虚荣度评分: 0/100 -> 健康

【累计注册用户数】虚荣度评分: 75/100 -> 疑似虚荣
  ! 上涨天数占比 100%（>70%）
  ! 最长连涨 89 期（>10）
  ! 与北极星r=0.11（<0.3）

【被活动污染的新增】虚荣度评分: 25/100 -> 健康
  ! 与北极星r=-0.02（<0.3）

============================================================
AARRR各环节虚荣指标 vs 健康指标
============================================================
获取: 新增用户数 -> 有效新增用户数(有核心行为)
激活: 注册完成率 -> 首日核心行为完成率
留存: MAU(月活) -> 同期群D30留存率
变现: GMV(总交易额) -> 毛利率/净收入
推荐: 邀请发送量 -> 邀请->注册转化率 x 被邀请人留存
```

概念群 C：[防护] 辛普森悖论在 AARRR 中的陷阱

> **深度类型**：[M][C] | 主课在 2.3 节一句话带过

🔴 认知冲突

快达的留存率 Q1->Q2「提升了」：Q1 D7留存=22%，Q2=25%。增长团队庆祝。

但你按渠道拆开——脊背发凉：

- SEO：Q1=30% -> Q2=27%（在恶化）
- 信息流：Q1=10% -> Q2=9%（也在恶化）
- 但Q2加大了SEO投放占比，从20%->50%

> 每个渠道的留存都在恶化，但总体留存「涨了」——因为渠道结构变了。这就是辛普森悖论。

In [5]:
```python
# [必修] 辛普森悖论在AARRR中的数值演示
import numpy as np; np.random.seed(42)

channels = ['SEO', '应用商店', '信息流', '社媒']
q1_users = np.array([8000, 15000, 40000, 12000])
q1_retention = np.array([0.30, 0.22, 0.10, 0.18])
q2_retention = np.array([0.27, 0.20, 0.09, 0.16])
q2_users = np.array([35000, 18000, 9000, 8000])

q1_overall = np.sum(q1_users * q1_retention) / np.sum(q1_users)
q2_overall = np.sum(q2_users * q2_retention) / np.sum(q2_users)
```

```
print("=" * 65)
print("辛普森悖论：每个渠道的留存都在恶化，总体留存却涨了")
print("=" * 65)
print(f"{'渠道':<10} {'Q1用户':>10} {'Q1留存':>8} {'Q2用户':>10} {'Q2留存':>8} {'留
print("-" * 68)
for i, ch in enumerate(channels):
    q1s = q1_users[i]/q1_users.sum()*100; q2s = q2_users[i]/q2_users.sum()*100
    print(f"{ch:<10} {q1_users[i]:>10,} {q1_retention[i]:>7.0%} {q2_users[i]:>10
print(f"\n总体D7留存：Q1={q1_overall:.1%} -> Q2={q2_overall:.1%} (变化：{q2_overa
print()
if q2_overall > q1_overall:
    print("!! 辛普森悖论触发！每个渠道都在恶化，但总体涨了")
    print(f"  原因：高质量渠道(SEO)用户占比从{q1_users[0]/q1_users.sum():.0%}升至{q
    print("  -> 如果你只看总体，你会以为留存优化成功了")
    print("  -> 正确结论：每个渠道的留存都在恶化，需要继续优化")
```

```
=================================================================
辛普森悖论：每个渠道的留存都在恶化，总体留存却涨了
=================================================================
渠道          Q1用户      Q1留存      Q2用户      Q2留存      留存变化      占比
变化
-------------------------------------------------------------------
SEO          8,000      30%      35,000      27%     -3.0%      +39.3pp
应用商店       15,000      22%      18,000      20%     -2.0%       +5.7pp
信息流        40,000      10%       9,000       9%     -1.0%      -40.5pp
社媒         12,000      18%       8,000      16%     -2.0%       -4.6pp

总体D7留存：Q1=15.8% -> Q2=21.6% (变化：+5.8%)

!! 辛普森悖论触发！每个渠道都在恶化，但总体涨了
  原因：高质量渠道(SEO)用户占比从11%升至50%
  -> 如果你只看总体，你会以为留存优化成功了
  -> 正确结论：每个渠道的留存都在恶化，需要继续优化
```

练习：识别辛普森悖论

快达的「新用户激活率」从 Q1=35% 涨到 Q2=42%。拆开看：

- Android：Q1=40% -> Q2=38%（在降）
- iOS：Q1=50% -> Q2=48%（在降）
- Q2 iOS 用户占比从 20%->60%

问题：辛普森悖论触发了吗？真正的激活优化该继续还是停？

参考答案

触发。每个平台的激活率都在降，但总体从 35%->42%——因为 iOS 占比从 20%->60%，iOS 激活率天然更高。

正确判断：激活优化没有效果（每个平台都在恶化）。渠道策略（多投 iOS）正确但不应掩盖激活优化本身的失败。

常见偏差 1：「总体涨了就是优化有效」-> 忽略了结构变化。**补救**：永远按关键维度拆解趋势。

常见偏差 2：「辛普森悖论只是统计学游戏」-> 错了。如果停掉激活优化团队（因为「总体已经涨了」），渠道结构稳定后真实激活率会暴露。**补救**：用同期群 x 渠道二维矩阵跟踪。

概念群 D：[落地] AARRR 漏斗的生产级 SQL 实现

> **深度类型**：[C] 代码深度 | 主课用 NumPy 演示，真实工作在 SQL 里算

[必修] 从 NumPy 到生产级 SQL

① **一句话白话**：主课用 NumPy 向量化算漏斗——适合探索。上生产需要 SQL——用 WITH CTE 链式构建漏斗，每一步存一张临时表，最后 JOIN 汇总。

```
-- ============================================================
-- [落地] AARRR 漏斗 -- 生产级 SQL（Hive/SparkSQL）
-- 用途：每周跑一次，输出全产品AARRR漏斗 + 各渠道漏斗
-- 参数：${dt}=分析日期，${window_days}=分析窗口
-- ============================================================

WITH
-- A层：获取 -- 各渠道过去N天的新注册用户
acquisition AS (
    SELECT channel, COUNT(DISTINCT user_id) AS new_users
    FROM dwd_user_register_di
    WHERE dt BETWEEN DATE_SUB('${dt}', ${window_days}) AND '${dt}'
    GROUP BY channel
),

-- A层：激活 -- 注册后7天内完成核心行为（非仅注册！）
activation AS (
    SELECT r.channel,
        COUNT(DISTINCT r.user_id) AS registered_users,
        COUNT(DISTINCT CASE
            WHEN a.core_action_cnt >= 1
              AND a.dt BETWEEN r.register_time AND
DATE_ADD(r.register_time, 7)
            THEN r.user_id END) AS activated_users
    FROM dwd_user_register_di r
    LEFT JOIN dwd_user_active_di a ON r.user_id = a.user_id
    WHERE r.dt BETWEEN DATE_SUB('${dt}', ${window_days}) AND '${dt}'
    GROUP BY r.channel
),

-- R层：留存（D7）
retention AS (
    SELECT r.channel,
        COUNT(DISTINCT r.user_id) AS cohort_users,
```

```
        COUNT(DISTINCT CASE WHEN a.dt = DATE_ADD(r.register_time, 7)
            THEN r.user_id END) AS d7_retained_users
    FROM dwd_user_register_di r
    LEFT JOIN dwd_user_active_di a ON r.user_id = a.user_id
    WHERE r.dt BETWEEN DATE_SUB('${dt}', ${window_days}+7) AND
DATE_SUB('${dt}', 7)
    GROUP BY r.channel
),

-- R层：变现
-- 注意：channel 取自 dwd_user_register_di（首次触点归因），非 active 表
revenue AS (
    SELECT COALESCE(r.channel, 'unknown') AS channel,
        COUNT(DISTINCT a.user_id) AS active_users_in_window,
        COUNT(DISTINCT p.user_id) AS paying_users,
        COALESCE(SUM(p.amount), 0) AS total_revenue
    FROM (SELECT DISTINCT user_id FROM dwd_user_active_di
          WHERE dt BETWEEN DATE_SUB('${dt}', ${window_days}) AND '${dt}')
a
    LEFT JOIN dwd_payment_di p ON a.user_id = p.user_id
        AND p.dt BETWEEN DATE_SUB('${dt}', ${window_days}) AND '${dt}'
    LEFT JOIN dwd_user_register_di r ON a.user_id = r.user_id
    GROUP BY COALESCE(r.channel, 'unknown')
),

-- R层：推荐
referral AS (
    SELECT COALESCE(r.channel, 'unknown') AS channel,
        COUNT(DISTINCT inviter_id) AS inviters,
        COUNT(DISTINCT CASE WHEN inv.status = 'registered' THEN invitee_id
END) * 1.0
            / NULLIF(COUNT(DISTINCT inviter_id), 0) AS k_factor
    FROM dwd_invite_log_di inv
    LEFT JOIN dwd_user_register_di r ON inv.inviter_id = r.user_id
    WHERE inv.dt BETWEEN DATE_SUB('${dt}', ${window_days}) AND '${dt}'
    GROUP BY COALESCE(r.channel, 'unknown')
)

-- 汇总：AARRR 漏斗全景
SELECT
    COALESCE(acq.channel, act.channel, ret.channel, rev.channel,
ref.channel) AS channel,
    acq.new_users AS A_获取_新用户,
    ROUND(act.activated_users*1.0 / NULLIF(act.registered_users,0), 4) AS
A_激活率,
    ROUND(ret.d7_retained_users*1.0 / NULLIF(ret.cohort_users,0), 4) AS
R_D7留存率,
    ROUND(rev.paying_users*1.0 / NULLIF(rev.active_users_in_window,0), 4)
AS R_付费转化率,
    ROUND(ref.k_factor, 4) AS R_K因子,
    rev.total_revenue AS R_总收入
FROM acquisition acq
FULL OUTER JOIN activation act ON acq.channel = act.channel
FULL OUTER JOIN retention ret ON COALESCE(acq.channel, act.channel) =
ret.channel
FULL OUTER JOIN revenue rev ON COALESCE(acq.channel, act.channel,
```

```
ret.channel) = rev.channel
FULL OUTER JOIN referral ref ON COALESCE(acq.channel, act.channel,
ret.channel, rev.channel) = ref.channel
ORDER BY A_获取_新用户 DESC;
```

③ 关键设计决策:

决策点	本SQL做法	为什么
激活窗口	注册后7天内	足够时间探索，但不能太长
留存窗口	D7精确匹配	D7是行业标准
变现归属	活跃用户中付费的	不活跃用户没有付费机会
K因子	简化版	生产环境应用精确时间窗口匹配
归因模型	首次触点	简单透明，适合渠道分析

概念群 E：[落地] LTV 的四种估算方法 + 代码对比

> **深度类型**：[M][C] | [选修] | 主课提到 LTV > CAC 是铁律，但没教 LTV 怎么算

In [6]:
```
# [选修] LTV的四种估算方法对比
import numpy as np; np.random.seed(42)

n_users, n_months = 1000, 12
true_ltv = np.random.lognormal(mean=3.5, sigma=1.2, size=n_users)
monthly_payments = np.zeros((n_users, n_months))

for u in range(n_users):
    active = True; base = true_ltv[u] / n_months * 2
    for m in range(n_months):
        if not active: break
        monthly_payments[u,m] = max(0, base*np.exp(-0.1*m) + np.random.randn()*2
        if np.random.random() < 0.15: active = False

actual = monthly_payments.sum(axis=1); actual_ltv = actual.mean()

# 方法1：ARPU x 平均生命周期
ltv1 = monthly_payments[monthly_payments>0].mean() * (monthly_payments>0).sum()/

# 方法2：前3月累计 x 放大系数
ltv2 = monthly_payments[:,:3].sum(axis=1).mean() * 2.8

# 方法3：几何级数求和 LTV=ARPU_month1/(1-r)
m1 = monthly_payments[:,0].mean()
r = np.mean(monthly_payments[:,1]>0) / max(np.mean(monthly_payments[:,0]>0), 0.0
ltv3 = m1 / (1 - min(r, 0.95))

# 方法4：同期群中位数LTV
ltv4 = np.percentile(actual, 50)

print("=" * 60)
```

```
print("LTV 四种估算方法对比")
print("=" * 60)
print(f"{'方法':<28} {'估算LTV':>10} {'vs真实值':>10}")
print("-" * 50)
for name, val in [('1.ARPU x 平均生命周期',ltv1), ('2.前3月累计 x 系数',ltv2),
    ('3.几何级数求和',ltv3), ('4.同期群中位数LTV',ltv4)]:
    print(f"{name:<28} {val:>9.1f}元 {(val-actual_ltv)/actual_ltv:>+9.1%}")
print(f"\n真实LTV = {actual_ltv:.1f}元")
print("\n关键结论: ")
print("  1.方法2(前3月x系数)是业界最常用--平衡了精度和数据可得性")
print("  2.方法4(同期群中位数)最精确--需6个月以上数据")
print("  3.永远用中位数而非均值--少数大R会把均值拉高")
```

```
============================================================
LTV 四种估算方法对比
============================================================
方法                                   估算LTV       vs真实值
--------------------------------------------------
1.ARPU x 平均生命周期                    82.7元      +0.0%
2.前3月累计 x 系数                      118.0元     +42.6%
3.几何级数求和                          130.2元     +57.4%
4.同期群中位数LTV                        56.6元     -31.5%

真实LTV = 82.7元

关键结论:
  1.方法2(前3月x系数)是业界最常用--平衡了精度和数据可得性
  2.方法4(同期群中位数)最精确--需6个月以上数据
  3.永远用中位数而非均值--少数大R会把均值拉高
```

概念群 F：[替代] Growth Loops——为什么「漏斗」思维不够用

> **深度类型**：[B] | [必修] | AARRR 是漏斗模型，但现代增长理论更强调「循环」

[必修] Growth Loops vs AARRR Funnel

① **一句话白话**：AARRR 是漏斗——进来一批用户，一路筛到底。Growth Loops 是循环——你的输出变成下一轮的输入，像滚雪球。漏斗是消耗型增长，Loop 是复利型增长。

② **思维差异**：

维度	AARRR 漏斗	Growth Loop
增长逻辑	用户进来->层层筛选->少数付费	用户进来->使用->产生内容/数据/邀请->吸引更多用户
投入产出	投放=花钱买用户(线性)	产品=用户创造用户(复利)
核心问题	哪个环节漏水最严重?	哪个循环转得最快?
优化方向	降低漏斗每步流失率	加速循环每次转速
数学表达	活跃=A0 x 激活率 x 留存率 x ...	活跃=A0 x (1+k+k^2+...) = A0/(1-k)

③ **四种经典 Growth Loop**：

Loop类型	循环逻辑	代表产品	快达能用吗?
内容Loop	用户产生内容->搜索收录->更多用户->更多内容	知乎/小红书	乘客写评价->评价吸引新乘客
病毒Loop	用户邀请好友->好友注册->新用户再邀请	WhatsApp/拼多多	已有(K=0.2)，但很低
付费Loop	投放获客->用户付费->收入再投放->更多用户	电商/游戏	快达核心Loop(LTV/CAC需>3)
数据Loop	用户行为->算法更准->体验更好->更多行为	抖音/Netflix	ETA预测->体验更好

④ **与核心概念的关系**：[替代] — AARRR 帮你找漏水的环节，Growth Loop 帮你想除了堵漏水还能怎么让增长飞起来。

概念群 G：[落地] AARRR 指标身份证

> **来源**：全课程永久空白区 Section 5.3 | **深度类型**：[B][C] | [选修] W1D2 主课讲了「好指标 5 条」，但没给指标的标准化文档模板。

In [7]:
```
# [选修] AARRR指标身份证 -- 模板 + 快达示例
class MetricIdentityCard:
    """指标身份证: 10字段标准化模板。锁死口径，版本可追溯。"""
    def __init__(self, name, name_cn, definition, formula, data_source,
                 frequency, owner, known_issues, sla, version):
        self.name=name; self.name_cn=name_cn; self.definition=definition
        self.formula=formula; self.data_source=data_source
        self.frequency=frequency; self.owner=owner
        self.known_issues=known_issues; self.sla=sla; self.version=version

    def print_card(self):
        print(f"{'='*50}\n指标身份证: {self.name_cn} ({self.name})\n{'='*50}")
        print(f"  定义: {self.definition}\n  公式: {self.formula}")
        print(f"  数据源: {self.data_source}\n  更新频率: {self.frequency}")
        print(f"  负责人: {self.owner}\n  SLA: {self.sla}")
        print(f"  已知问题: {self.known_issues}\n  版本: v{self.version}\n")

print("快达出行 -- AARRR 指标身份证库\n")
cards = [
    MetricIdentityCard('new_valid_users','有效新增用户数',
        '注册后7天内至少完成1次核心行为(下单)的去重新用户',
        'COUNT(DISTINCT user_id) WHERE 注册时间在窗口内 AND 7天内完成>=1次下单',
        'dwd_user_register_di + dwd_trip_core_di','每日','增长分析师-张三',
        '渠道归因使用首次触点，可能存在跨渠道用户误归因','每日10:00前','2.1'),
    MetricIdentityCard('activation_rate_d7','D7激活率',
        '注册后7天内完成>=1次完单的用户占同期新注册用户的比例',
        '激活用户数(注册7天内完单>=1) / 同期注册用户总数',
        'dwd_user_register_di + dwd_trip_core_di(event_type=order_complete)',
        '每日(T+8产出)','产品分析师-李四',
```

```
        '受节假日效应影响，周同比比日环比更有意义','每日11:00前','1.3'),
    MetricIdentityCard('d30_retention_rate','D30留存率',
        '注册30天后仍活跃(当日有session)的用户占同期群比例',
        'D30当天活跃用户数 / 同期群注册用户总数',
        'ads_user_cohort_retention','每日(T+31产出)','数据产品-王五',
        '依赖离线ETL延迟2小时，节假日可滞后1天','每日12:00前','3.0'),
    MetricIdentityCard('k_factor_30d','30天K因子',
        '过去30天内每个活跃用户通过邀请机制平均带来的有效新用户数',
        '邀请成功注册的新用户数 / 同期发起邀请的活跃用户数',
        'dwd_invite_log_di + dwd_user_active_di','每周','增长分析师-张三',
        '仅覆盖有邀请码机制的推荐，自然口碑无法追踪','每周一12:00前','1.0'),
]
for card in cards: card.print_card()
```

```
快达出行 -- AARRR 指标身份证库

==================================================
指标身份证：有效新增用户数 (new_valid_users)
==================================================
  定义：注册后7天内至少完成1次核心行为(下单)的去重新用户
  公式：COUNT(DISTINCT user_id) WHERE 注册时间在窗口内 AND 7天内完成>=1次下单
  数据源：dwd_user_register_di + dwd_trip_core_di
  更新频率：每日
  负责人：增长分析师-张三
  SLA：每日10:00前
  已知问题：渠道归因使用首次触点，可能存在跨渠道用户误归因
  版本：v2.1

==================================================
指标身份证：D7激活率 (activation_rate_d7)
==================================================
  定义：注册后7天内完成>=1次完单的用户占同期新注册用户的比例
  公式：激活用户数(注册7天内完单>=1) / 同期注册用户总数
  数据源：dwd_user_register_di + dwd_trip_core_di(event_type=order_complete)
  更新频率：每日(T+8产出)
  负责人：产品分析师-李四
  SLA：每日11:00前
  已知问题：受节假日效应影响，周同比比日环比更有意义
  版本：v1.3

==================================================
指标身份证：D30留存率 (d30_retention_rate)
==================================================
  定义：注册30天后仍活跃(当日有session)的用户占同期群比例
  公式：D30当天活跃用户数 / 同期群注册用户总数
  数据源：ads_user_cohort_retention
  更新频率：每日(T+31产出)
  负责人：数据产品-王五
  SLA：每日12:00前
  已知问题：依赖离线ETL延迟2小时，节假日可滞后1天
  版本：v3.0

==================================================
指标身份证：30天K因子 (k_factor_30d)
==================================================
  定义：过去30天内每个活跃用户通过邀请机制平均带来的有效新用户数
  公式：邀请成功注册的新用户数 / 同期发起邀请的活跃用户数
  数据源：dwd_invite_log_di + dwd_user_active_di
  更新频率：每周
  负责人：增长分析师-张三
  SLA：每周一12:00前
  已知问题：仅覆盖有邀请码机制的推荐，自然口碑无法追踪
  版本：v1.0
```

第四部分：综合沙盘演练

综合练习 1：多概念协同——激活提升的飞轮放大

场景：CEO问「如果激活从35%拉到50%，对明年收入的影响到底多大？」

任务：用三个工具协同——飞轮数学模型 + 虚荣指标检测器 + LTV估算

In [8]:
```
# 综合练习1：激活提升的飞轮放大 + 指标可靠性检验
import numpy as np; np.random.seed(42)

def calc_annual(params):
    total_acq = params['acq'] / (1 - params['k'])
    activated = total_acq * params['act']
    paying = activated * params['ret'] * params['pay']
    monthly = paying * params['arpu']
    cost = params['acq'] * params['cac'] * 12
    return {'income': monthly*12, 'profit': monthly*12-cost, 'paying': paying}

base = calc_annual({'acq':200000,'act':0.35,'ret':0.12,'pay':0.06,'arpu':80,'k':
imp = calc_annual({'acq':200000,'act':0.50,'ret':0.12,'pay':0.06,'arpu':80,'k':0

print("=" * 60)
print("综合练习1：激活从35%->50%对快达明年收入的影响")
print("=" * 60)
for k,label in [('income','年收入'),('profit','年利润'),('paying','月付费用户')]:
    print(f"{label}: {base[k]:>15,.0f} -> {imp[k]:>15,.0f}  ({imp[k]/base[k]-1:>
print(f"\n激活对收入弹性：{(imp['income']/base['income']-1)/0.15:.2f}")
print("结论：激活+15ppt -> 年收入+43%（含飞轮放大）")

# 验证激活指标有效性
n=10000; levels=np.random.choice(['only_reg','1_ride','3_rides'],n,p=[0.3,0.5,0.
ret30=np.where(levels=='only_reg',np.random.binomial(1,0.03,n),
      np.where(levels=='1_ride',np.random.binomial(1,0.15,n),np.random.binomial(
print("\n【激活指标有效性检验】")
for lv,label in [('only_reg','仅注册'),('1_ride','完单>=1次'),('3_rides','完单>=3
    m=levels==lv; print(f"{label}: 用户{m.sum():,}, D30留存{ret30[m].mean():.1%}"
print("-> 注册=虚荣(留存3%)，完单>=1次=健康(留存15%)")
```

```
============================================================
综合练习1：激活从35%->50%对快达明年收入的影响
============================================================
年收入:         604,800 ->         864,000  (+42.9%)
年利润:     -35,395,200 ->     -35,136,000  (-0.7%)
月付费用户:             630 ->             900  (+42.9%)

激活对收入弹性：2.86
结论：激活+15ppt -> 年收入+43%（含飞轮放大）

【激活指标有效性检验】
仅注册: 用户3,053, D30留存3.4%
完单>=1次: 用户4,985, D30留存14.6%
完单>=3次: 用户1,962, D30留存34.9%
-> 注册=虚荣(留存3%)，完单>=1次=健康(留存15%)
```

综合练习 2：沙盘变体——快达进军货运

场景：快达拓展货运业务（同城小件配送）。出行高频，货运低频且B端为主。

任务：AARRR每个环节定义怎么改写？激活的「啊哈时刻」是什么？K因子比出行高还是低？

参考答案

环节	出行	货运
获取	乘客下载App	小商家入驻（B端获取 > C端）
激活	完成第一单	首单准时送达（核心价值=准时）
留存	持续打车	月发件>=3次=活跃商家
变现	乘客付费	运费抽成+增值服务(保险/包装)
推荐	邀请好友打车	B端商家间转介绍(同行群/商会)

K因子判断：货运的K比出行更低——低频、B端更理性、寄件不是社交场景。增长更多靠付费Loop而非病毒Loop。

综合练习 3：边界判断——什么场景不该用 AARRR?

#	场景	用AARRR?	替代框架	原因
1	评估App首页改版是否提升体验	❌	HEART	改版评估=体验而非增长漏斗
2	判断成都vs合肥哪个优先补贴司机	⚠️	AARRR+供需模型	补贴该给谁=供需四象限问题
3	首单用券vs免起步价哪个ROI高	⚠️	AARRR+AB实验	识别了环节，策略选择需AB实验
4	司机端新增语音播报对效率影响	❌	HEART(任务成功)	效率=Task Success，非增长漏斗

> **判断原则**：AARRR擅长回答「增长健康吗、卡在哪」，不擅长回答「哪个方案更好」和「体验好不好」。前者用AB实验，后者用HEART。

综合练习 4：落地——从概念到可交付产物

场景：明天给CEO汇报AARRR健康诊断。自检清单：

- [ ] AARRR漏斗SQL中，激活的定义是「完单>=1次」而非「注册」
- [ ] 每个环节指标都有指标身份证（至少填定义/公式/数据源三字段）
- [ ] 飞轮模型区分了投放用户和推荐用户的收入贡献

- [ ] 用辛普森悖论检测验证了「总体留存涨是否因渠道结构变了」

第五部分：知识网络图

本拓展概念全景

概念群	概念	关系类型	必修/选修	深度类型	来源
核心	飞轮数学建模	核心深化	必修	[M][C]	主课2.6/2.11深化
A	HEART vs AARRR	并行	必修	[B]	永久空白区Sec 5.3
B	虚荣指标检测器	防护	必修	[C]	永久空白区Sec 5.3
C	辛普森悖论陷阱	防护	必修	[M][C]	主课引子(2.3节)
D	生产级漏斗SQL	落地	必修	[C]	主课NumPy->SQL
E	LTV四种估算	落地	选修	[M][C]	技能速查表
F	Growth Loops	替代	必修	[B]	行业实践
G	指标身份证	落地	选修	[B][C]	永久空白区Sec 5.3

三版本覆盖对比

知识域	主课程(W2D1)	本拓展新增
AARRR五环节	✅ 详细	C端/B端改写
飞轮效应	✅ 定性	✅ 数学建模+代码
漏斗计算	✅ NumPy	✅ 生产级SQL
K因子	✅ 定义	✅ 因子分解+延迟建模
LTV/CAC	✅ 提及	✅ 四种算法+代码
HEART	一行	✅ 系统对比+选择矩阵
虚荣指标	W1D2定性	✅ 自动检测器
辛普森悖论	一句话	✅ 完整数值演示
Growth Loops	--	✅ 新增
指标身份证	--	✅ 新增

跨周连接

```
W1D2(好指标5条) -> 本拓展B(虚荣指标检测器) -> W2D5(RFM)
W1D3(OSM)       -> 本拓展D(漏斗SQL)        -> W5D3(窗口函数)
W1D4(CMOP)      -> 本拓展A(飞轮建模)       -> W3D3(补贴ROI)
W1D5(指标拆解)  -> 本拓展C(辛普森悖论)     -> W6D3(面试SQL)
```

第六部分：元认知自评

学习循环自评表

我能……	还不能	差不多	完全能
用飞轮公式 A0/(1-k) 计算K因子对投放的放大倍数	□	□	□
用虚荣指标检测器的四个检验判断指标是否健康	□	□	□
识别AARRR数据中的辛普森悖论	□	□	□
写出生产级AARRR漏斗SQL（CTE链式+渠道维度+归因逻辑）	□	□	□
说清HEART和AARRR的适用场景区别	□	□	□
用LTV四种方法估算用户价值	□	□	□
区分漏斗思维(AARRR)和循环思维(Growth Loops)	□	□	□
为AARRR每个环节的指标填指标身份证	□	□	□

多维度自评（0=完全不会, 1=看懂了写不出, 2=能独立写出并解释）

维度	0	1	2	证据
数学建模能力	□	□	□	
代码实现能力	□	□	□	
业务诊断能力	□	□	□	
框架选择能力	□	□	□	
SQL工程能力	□	□	□	

维度	0	1	2	证据
指标设计能力	□	□	□	

沙盘复述测试

用 3 分钟向一个不懂 AARRR 的同事解释：

1. 快达的 AARRR 漏斗长什么样？
2. 它卡在哪？
3. 你建议怎么做？为什么？

如果 3 分钟内讲不清楚 -> 你对 AARRR 的理解还不够深（只记住了五个字母，没有内化成诊断工具）。

03analyst_thinking · Week 2 Day 1 拓展 · v4.1 深度网络版 · 2026-06-24